

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	N.Adi Narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	18-08-2026

Code of Conduct

I Adi Narayana (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Adi_____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform **Reference Resolution** by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

2. Develop an algorithm to analyze **Text Coherence** and **Discourse Structure** by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

3. Design an algorithm for a **Conversational Agent** that identifies **Dialogue Acts** from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"

Agent: "Sure, where would you like to travel?"

User: "I want to go to Chennai."

Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

4. Design an algorithm for **Language Generation** that converts a semantic representation into a grammatically correct sentence through **Surface Realization**. Consider the semantic input:

Action: Buy
Agent: Student
Object: Book
Tense: Past

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

5. Design an algorithm that combines **Interlingua-based Machine Translation** and **Statistical Translation Approaches**. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. Analyze the source sentence.
2. Create an Interlingua representation.
3. Generate candidate target-language translations.

4. Apply statistical scoring to select the most appropriate translation.
5. Produce the final translated sentence.

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

Q1. Reference Resolution – Entity Linking Algorithm

Design an algorithm to perform Reference Resolution by identifying pronouns and linking them to their correct entities in a discourse. Given: “Ravi met Arun at the library. He borrowed a book and later returned it.”

1. Problem Understanding

Reference resolution identifies which previously mentioned entity a referring expression or pronoun refers to. The referring expressions here are He and it; the entities introduced before them are Ravi, Arun, library and book. The system must determine He → Ravi or Arun, and it → book or another object.

2. Input and Output

Input: Ravi met Arun at the library.
He borrowed a book and later returned it.

Output: Ravi met Arun at the library.
Ravi borrowed a book and later returned the book.

3. Constraints Used

Constraint	Purpose
Gender agreement	Checks masculine/feminine compatibility
Number agreement	Checks singular/plural compatibility
Recency	Prefers recently mentioned entities
Grammatical role	Considers subject/object positions
Semantic compatibility	Checks whether the action makes sense
Discourse coherence	Maintains a consistent entity chain

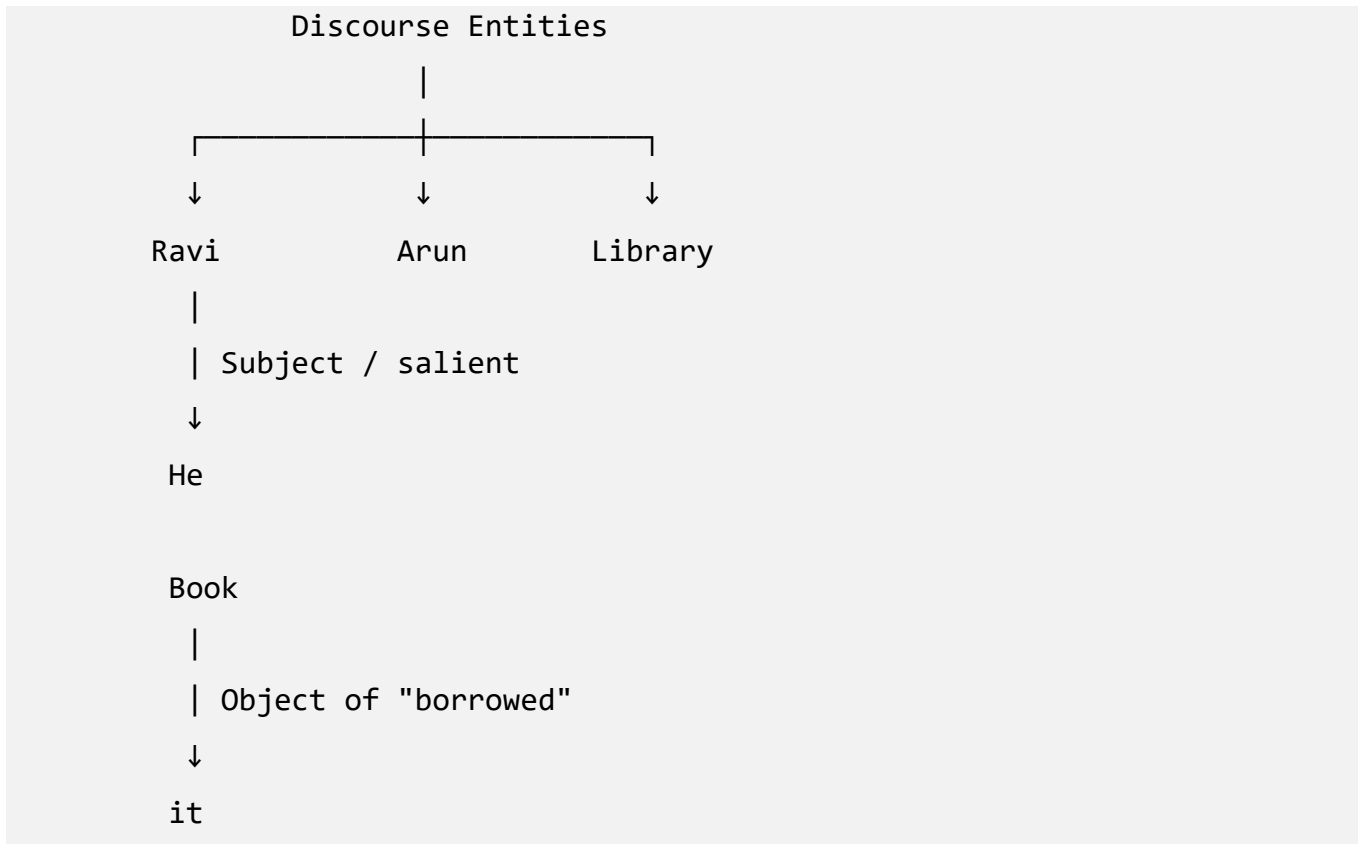
Both Ravi and Arun are masculine and singular, so gender and number alone cannot distinguish He — discourse context and grammatical salience become important.

4. Reference Resolution Process

From sentence 1, entities E1=Ravi, E2=Arun, E3=Library are introduced. Sentence 2 has pronouns P1=He and P2=it, with candidates {Ravi, Arun} for P1 and {Ravi, Arun, library, book} for P2. For He, Ravi

is preferred since he is the subject of the previous sentence and the most salient entity. For it, semantic compatibility strongly selects book, since a person can borrow and return a book.

5. Entity Linking Representation



Therefore: He → Ravi, it → Book.

6. Pseudocode

ALGORITHM ReferenceResolution(Text)

INPUT: Text containing sentences and referring expressions

OUTPUT: Resolved discourse with pronouns replaced

BEGIN

Sentences ← SplitIntoSentences(Text)

Entities ← EmptyList()

ResolvedText ← EmptyList()

FOR each Sentence in Sentences DO

 Tokens ← Tokenize(Sentence)

 Pronouns ← IdentifyPronouns(Tokens)

```

FOR each Pronoun in Pronouns DO
    Candidates ← FindPreviousEntities(Entities)

    FOR each Candidate in Candidates DO
        Score ← 0
        IF GenderMatches(Pronoun, Candidate) THEN
            Score ← Score + 2
        END IF
        IF NumberMatches(Pronoun, Candidate) THEN
            Score ← Score + 2
        END IF
        IF IsRecent(Candidate) THEN
            Score ← Score + 1
        END IF
        IF SemanticCompatible(Pronoun, Candidate, Sentence) THEN
            Score ← Score + 3
        END IF
        IF IsDiscourseSalient(Candidate) THEN
            Score ← Score + 2
        END IF
    END FOR

    BestCandidate ← CandidateWithHighestScore()
    ReplacePronoun(Pronoun, BestCandidate)
END FOR

AddNewEntities(Sentence, Entities)
AddResolvedSentence(Sentence, ResolvedText)
END FOR

RETURN ResolvedText
END

```

7. Applying the Algorithm

Step 1 – “Ravi met Arun at the library” introduces entities Ravi, Arun, Library.

Step 2 – Detecting He: candidates {Ravi, Arun}, both masculine singular; Ravi is the subject and Arun the object, so Ravi receives higher discourse salience — He → Ravi.

Step 3 – Detecting it: candidates {Ravi, Arun, Library, Book}; semantic compatibility rules out borrowing a person or, less directly, a library, leaving borrow Book as valid — it → Book.

Validation

Pronoun	Selected Entity	Reason
He	Ravi	Subject salience + masculine singular
it	Book	Semantic compatibility + object role

The algorithm successfully uses discourse context, grammatical information, and semantic compatibility.

Conclusion

Reference resolution allows an NLP system to understand that different expressions may refer to the same entity. In this example, “He” is resolved to Ravi using discourse salience and “it” is resolved to the book using semantic compatibility, preventing ambiguity and enabling a coherent discourse representation for downstream NLP systems.

Q2. Text Coherence and Discourse Structure

Design an algorithm that identifies logical relationships between sentences such as cause-effect, sequence and elaboration. Given: “The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online.”

1. Problem Understanding

Text coherence refers to the logical and meaningful connection between different parts of a text. The three sentences are connected through a clear chain: heavy rainfall → roads flooded → schools closed → students attended classes online. The main relationships are cause, effect, consequence and sequence.

2. Sentence Identification

S1 = The roads were flooded after heavy rainfall.

S2 = Therefore, schools were closed for the day.

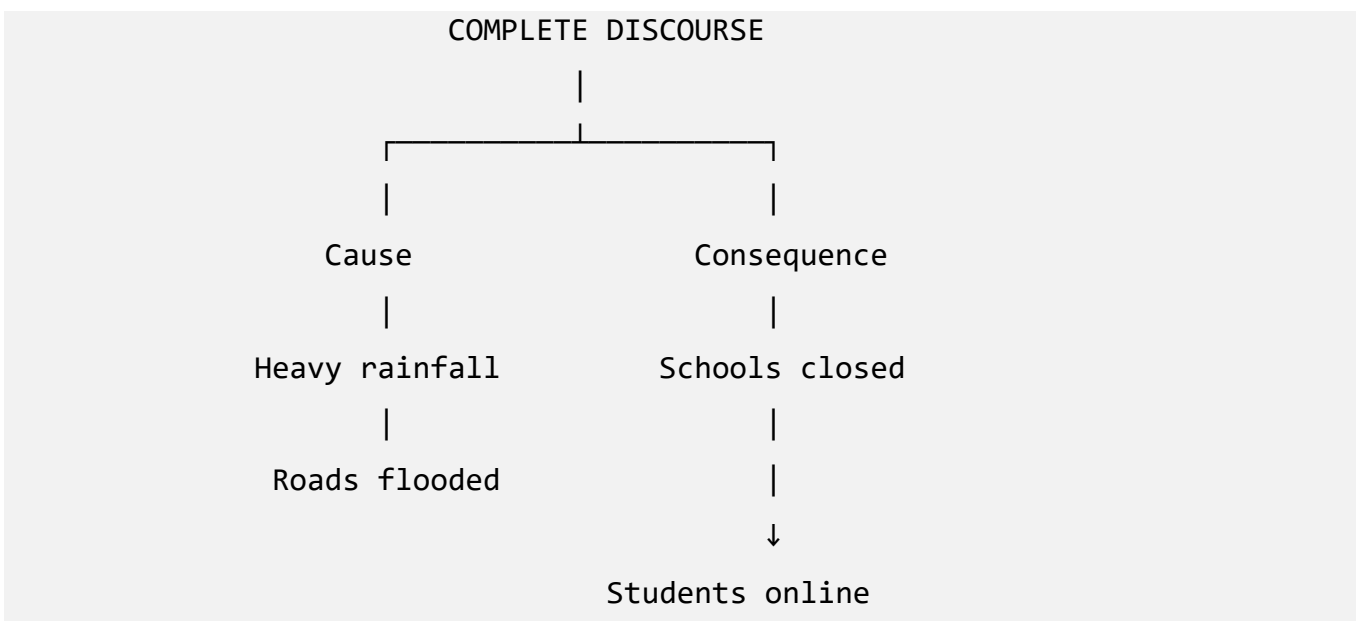
S3 = Students attended classes online.

3. Discourse Relations

S1 → S2: the flooding caused disruption, leading to school closure (S1 = Cause, S2 = Effect). The word “Therefore” is a strong discourse marker indicating consequence.

S2 → S3: after schools closed, students attended classes online (S2 = Situation, S3 = Response/Consequence) — a sequential relationship.

4. Discourse Structure



More simply:

```
S1
|
| Cause
↓
S2
|
| Consequence / Sequence
↓
S3
```

5. Pseudocode

ALGORITHM CoherenceAnalysis(Text)

INPUT: A sequence of sentences

OUTPUT: Discourse relations and coherence structure

BEGIN

Sentences ← SplitIntoSentences(Text)

FOR i ← 1 TO Length(Sentences) - 1 DO

Current ← Sentences[i]

Next ← Sentences[i + 1]

Markers ← FindDiscourseMarkers(Current, Next)

IF ContainsCauseMarker(Markers)

OR IndicatesCause(Current, Next) THEN

Relation ← "CAUSE-EFFECT"

ELSE IF ContainsSequenceMarker(Markers)

OR IndicatesTemporalOrder(Current, Next) THEN

Relation ← "SEQUENCE"

ELSE IF IndicatesExplanation(Current, Next) THEN

Relation ← "ELABORATION"

ELSE

Relation ← "UNKNOWN"

```

        END IF

        StoreRelation(Current, Next, Relation)
    END FOR

    BuildDiscourseTree()
    EvaluateCoherence()

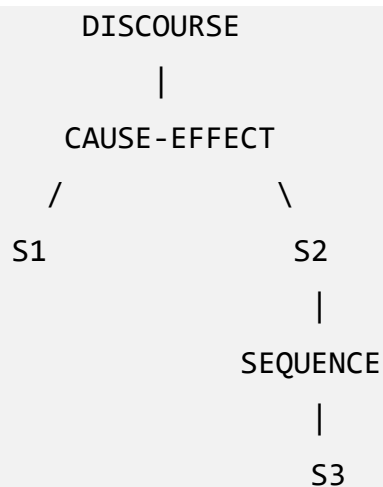
    RETURN DiscourseTree
END

```

6. Applying the Algorithm

S1 gives Event 1 = Heavy rainfall, Event 2 = Roads flooded, a Cause-Effect relation. S2's marker “therefore” explicitly indicates a consequence: road flooding → school closure. S3 provides the next event resulting from the closure: schools closed → online classes.

7. Discourse Tree



8. Coherence Evaluation

Sentence Pair	Relation	Evidence
S1 → S2	Cause-Effect	Flooding causes disruption; “Therefore”
S2 → S3	Consequence/Sequence	School closure leads to online classes

The discourse is coherent because every sentence contributes to the same situation.

Q3. Dialogue Act and Intent Recognition Algorithm

Design an algorithm for a conversational agent that identifies Dialogue Acts from user interactions, using the categories Request, Question, Inform, Confirmation and Action.

1. Dialogue Act Identification

Speaker	Utterance	Dialogue Act
User	Can you book a train ticket for me?	Request
Agent	Sure, where would you like to travel?	Question
User	I want to go to Chennai.	Inform
Agent	Your ticket has been booked.	Confirmation / Action

2. Intent Representation

```
Intent = BookTrainTicket
Intent = RequestDestination
Destination = Chennai
Action = BookTicket
Confirmation = TicketBooked
```

3. Pseudocode

```
ALGORITHM DialogueActRecognition(Dialogue)

INPUT: Sequence of user and agent utterances
OUTPUT: Dialogue-act sequence and detected user intent

BEGIN
    Acts ← EmptyList()
    Context ← EmptyContext()

    FOR each Utterance in Dialogue DO
        Tokens ← Tokenize(Utterance)
```

```

    IF ContainsRequestPattern(Utterance) THEN
        Act ← "REQUEST"
    ELSE IF ContainsQuestionPattern(Utterance) THEN
        Act ← "QUESTION"
    ELSE IF ContainsInformation(Utterance) THEN
        Act ← "INFORM"
    ELSE IF IndicatesCompletedAction(Utterance) THEN
        Act ← "CONFIRMATION"
    ELSE IF IndicatesSystemAction(Utterance) THEN
        Act ← "ACTION"
    ELSE
        Act ← "UNKNOWN"
    END IF

    UpdateContext(Context, Utterance, Act)
    Add(Acts, Act)

    IF Act = "REQUEST" THEN
        Intent ← IdentifyIntent(Utterance)
    END IF
END FOR

RETURN Acts, Intent, Context
END

```

4. Applying the Algorithm

“Can you book a train ticket for me?” — the pattern “Can you...?” combined with “book a train ticket” indicates a Request, giving Act = REQUEST, Intent = BOOK_TRAIN_TICKET.

“Sure, where would you like to travel?” asks for missing information: Act = QUESTION, RequiredEntity = Destination.

“I want to go to Chennai” provides the requested information: Act = INFORM, Destination = Chennai.

“Your ticket has been booked” indicates the requested action is complete: Act = CONFIRMATION, Action = BOOK_TICKET. The actual booking operation is an Action, while the utterance itself functions as a Confirmation.

5. Dialogue-Act Sequence

REQUEST → QUESTION → INFORM → ACTION / CONFIRMATION

6. State Representation

Intent: BookTrainTicket

Destination: Chennai

Status: Booked

This is useful because the agent does not need to ask the user again for information already provided.

7. Why Dialogue Acts Help

Dialogue acts help the agent determine what the user is trying to accomplish: Request signals a task to perform, Question signals a need for information, Inform adds information to context, Action performs an operation, and Confirmation tells the user the operation is complete.

Conclusion

The dialogue sequence REQUEST → QUESTION → INFORM → ACTION/CONFIRMATION allows the conversational agent to track the user's goal, collect missing information, and perform the required operation.

Q4. Language Generation and Surface Realization

Design an algorithm that converts a semantic representation into a grammatically correct natural language sentence through Surface Realization. Given semantic input: Action = Buy, Agent = Student, Object = Book, Tense = Past.

1. Semantic Representation

```
BUY(  
    Agent = Student,  
    Object = Book,  
    Tense = Past  
)
```

2. Lexical Selection

Semantic Concept	Selected Word
Agent	student
Action	bought
Object	book
Tense	past

The verb buy changes to bought because the tense is past.

3. Sentence Structure

English generally uses Subject + Verb + Object, so “Student + bought + book,” with determiners inserted to give “The + student + bought + a + book.”

4. Pseudocode

```
ALGORITHM SurfaceRealization(SemanticInput)
```

```
INPUT: Semantic representation containing  
       Action, Agent, Object and Tense
```

```
OUTPUT: Grammatically correct sentence
```

```
BEGIN
```

```
    Agent ← SemanticInput.Agent
```

```

Action ← SemanticInput.Action
Object ← SemanticInput.Object
Tense ← SemanticInput.Tense

AgentWord ← SelectLexicalItem(Agent)
ActionWord ← SelectLexicalItem(Action)
ObjectWord ← SelectLexicalItem(Object)

IF Tense = "Past" THEN
    ActionWord ← ConvertToPast(ActionWord)
ELSE IF Tense = "Present" THEN
    ActionWord ← ConvertToPresent(ActionWord)
END IF

AgentPhrase ← AddDeterminer(AgentWord)
ObjectPhrase ← AddDeterminer(ObjectWord)

Sentence ← AgentPhrase + " " + ActionWord + " " + ObjectPhrase
Sentence ← CapitalizeFirstLetter(Sentence)
Sentence ← AddFullStop(Sentence)

IF CheckGrammar(Sentence) = TRUE THEN
    RETURN Sentence
ELSE
    RETURN RegenerateSentence()
END IF
END

```

5. Applying the Algorithm

Semantic input: Agent = Student, Action = Buy, Object = Book, Tense = Past. Lexical selection maps Student → student, Buy → buy, Book → book. Tense transformation converts buy → bought. Sentence structuring gives Subject = The student, Verb = bought, Object = a book. Surface realization produces the final output.

“The student bought a book.”

6. Grammatical Validation

Property	Result
Subject present	✓
Verb present	✓
Object present	✓
Past tense	✓
Subject–verb agreement	✓
Determiners	✓
Word order	✓
Punctuation	✓

The sentence follows Determiner + Noun + Verb(Past) + Determiner + Noun, so the output is grammatically valid.

7. NLG Pipeline

Semantic Representation

↓

Lexical Selection

↓

Grammatical Planning

↓

Sentence Structuring

↓

Morphological Realization

↓

Surface Form

↓

Grammar Checking

↓

Final Sentence

Q5. Interlingua and Statistical Machine Translation

Design an algorithm combining Interlingua-based Machine Translation and Statistical Translation Approaches. Source sentence: "The boy is playing football."

1. Source Sentence Analysis

Tokens: The | boy | is | playing | football

Subject: The boy

Verb: is playing

Object: football

Agent = boy

Action = play

Activity = football

Tense/Aspect = Present Continuous

2. Interlingua Representation

An Interlingua is a language-independent semantic representation:

```
PLAY(  
    Agent = BOY,  
    Activity = FOOTBALL,  
    Tense = PRESENT,  
    Aspect = CONTINUOUS  
)
```

The key idea is that this semantic representation is independent of the source and target languages.

3. Translation Architecture

English Sentence

↓

Source Analysis

↓

Semantic Interpretation

↓

Interlingua

↓

Candidate Generation

↓

Statistical Scoring

↓

Best Candidate

↓

Target Language Surface Realization

↓

Final Translation

4. Pseudocode

ALGORITHM HybridTranslation(SourceSentence)

INPUT: Source sentence

OUTPUT: Final translated sentence

BEGIN

 Tokens ← Tokenize(SourceSentence)

 Syntax ← AnalyzeSyntax(Tokens)

 Semantic ← ExtractSemantics(Syntax)

 Interlingua ← CreateInterlingua(
 Agent, Action, Activity, Tense, Aspect
)

 Candidates ← GenerateTargetCandidates(Interlingua)

 BestScore ← $-\infty$

 BestCandidate ← NULL

 FOR each Candidate in Candidates DO

 TranslationScore ←

 LanguageModelScore(Candidate)

 + SemanticSimilarity(Candidate, Interlingua)

 + TranslationModelScore(Candidate)

```

    IF TranslationScore > BestScore THEN
        BestScore ← TranslationScore
        BestCandidate ← Candidate
    END IF
END FOR

FinalTranslation ← SurfaceRealize(BestCandidate)

RETURN FinalTranslation
END

```

5. Candidate Generation

With target language Tamil, the Interlingua (Agent = Boy, Action = Playing, Activity = Football, Aspect = Continuous) can generate several candidate translations, of which the first most accurately preserves the present continuous meaning.

1. சிறுவன் கால்பந்து விளையாடிக் கொண்டிருக்கிறான்.
2. பையன் கால்பந்து விளையாடுகிறான்.
3. சிறுவன் கால்பந்து விளையாடினான்.

6. Statistical Scoring

Scores are assigned based on translation model probability, language model probability, semantic similarity, and grammatical correctness.

Candidate	Semantic Score	Language Score	Overall
Candidate 1	0.96	0.94	0.95
Candidate 2	0.88	0.92	0.90
Candidate 3	0.60	0.91	0.76

Best Candidate = Candidate 1.

7. Final Translation

“சிறுவன் கால்பந்து விளையாடிக் கொண்டிருக்கிறான்.” — meaning “The boy is playing football,” preserving Boy + Playing + Football + present continuous aspect.

8. How Interlingua Reduces Ambiguity

Interlingua separates meaning representation from the surface form of the source language: English → Interlingua → Target Language. Instead of directly mapping every English word to a target-language word, the system first determines the meaning, reducing problems from word order, grammatical differences, morphological differences, tense/aspect, and lexical variation.

9. Role of Statistical Translation

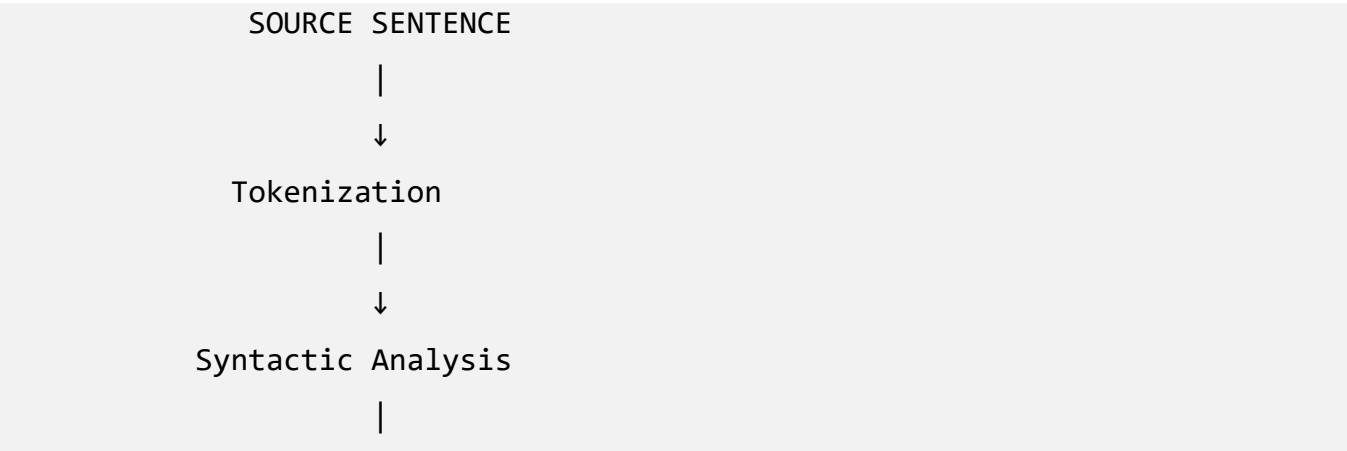
The statistical component selects the most natural target-language expression among multiple candidates by comparing $P(\text{Target} \mid \text{Source})$ or a combined score of translation probability, language model probability, and semantic compatibility. The candidate with the highest score becomes the final output.

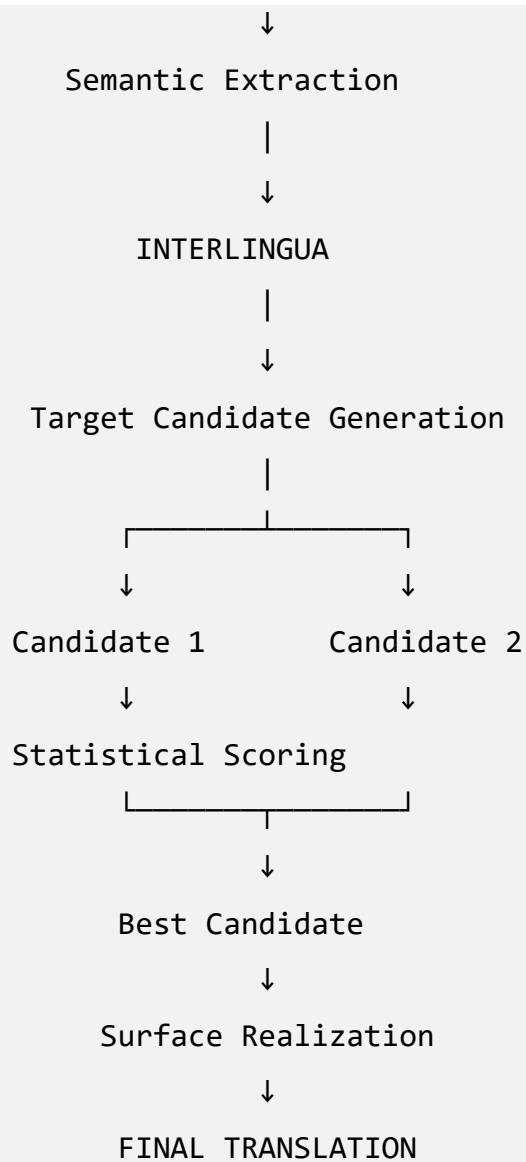
10. Advantages of the Hybrid Approach

Feature	Interlingua	Statistical Method
Meaning representation	Strong	Indirect
Language independence	High	Depends on training
Ambiguity handling	Explicit	Probability-based
Candidate selection	Limited	Strong
Natural fluency	Moderate	High
Semantic preservation	High	Variable
Adaptability	High	Data dependent

Combining both approaches provides the advantages of semantic representation and statistical ranking.

11. Complete Hybrid Workflow





12. Evaluation

Semantic preservation: the final translation preserves Agent = boy, Action = play, Activity = football, Aspect = continuous. Grammatical preservation: the target sentence correctly expresses an ongoing action. Ambiguity reduction: Interlingua first converts the source into a structured meaning representation. Fluency: statistical scoring helps select the most natural target-language expression. The combination therefore provides both semantic accuracy and natural fluency.

Overall Summary of the Five Algorithms

Q	Problem	Main Algorithm	Final Result
Q1	Reference Resolution	Entity Linking Algorithm	He → Ravi, it → Book
Q2	Text Coherence	Coherence Analysis Algorithm	Cause-Effect + Sequence
Q3	Dialogue Acts	Intent Recognition Algorithm	Request → Question → Inform → Confirmation
Q4	Language Generation	NLG Surface Realization	The student bought a book.
Q5	Machine Translation	Hybrid Translation Workflow	Tamil translation using Interlingua + statistical scoring

Final Conclusion

The five algorithms demonstrate how NLP systems can transform unstructured language into structured computational representations and then generate meaningful outputs.

Reference Resolution identifies entities and resolves pronouns using discourse context, grammatical compatibility, semantic compatibility and salience. Text Coherence Analysis identifies logical relationships such as cause-effect and sequence and represents them using a discourse structure.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWERS